Modelamiento en Física Computacional

Taller Práctico

Solución Numérica de Ecuaciones de Poisson y Laplace

Diferencias Finitas + Iteración de Jacobi

Comparativa: Fortran · C++ · Python (numpy)

Mallas 100×100 y 500×500 — Doble precisión

Marzo 2026

Contenido

1. Problemas Físicos
2. Método Numérico
3. Implementaciones
4. Resultados — Soluciones
5. Benchmark
6. Análisis
7. Conclusiones

Ecuaciones de Poisson y Laplace

Forma general (2D)

$$\nabla^2 V = \frac{\partial^2 V}{\partial x^2} + \frac{\partial^2 V}{\partial y^2} = f(x, y)$$

- $f = 0 \quad \rightarrow$ **Laplace**
- $f \neq 0 \quad \rightarrow$ **Poisson**

Aplicaciones

- Electrostática
- Conducción de calor (régimen permanente)
- Flujo potencial

Objetivo del taller

Resolver **4 EDPs** con solución analítica conocida en tres lenguajes y comparar rendimiento.

Lenguajes comparados

Lenguaje	Estrategia
Fortran	Bucles DO, arrays contig.
C++	`vector<vector<double»`
Python	numpy vectorizado

Los Cuatro Problemas

E1 — Poisson exponencial

$\nabla^2 V = (x^2 + y^2)e^{xy}, \quad [0,2] \times [0,1]$

CC: $V(0,y) = 1, \;\; V(2,y) = e^{2y}, \;\; V(x,0) = 1,$
$V(x,1) = e^x$

Analítica: $V = e^{xy}$

E2 — Laplace

$\nabla^2 V = 0, \quad [1,2] \times [0,1]$

CC: $V(1,y) = \ln(y^2+1), \;\; V(2,y) = \ln(y^2+4),$
$V(x,0) = 2\ln x, \;\; V(x,1) = \ln(x^2+1)$

Analítica: $V = \ln(x^2 + y^2)$

E3 — Poisson fuente constante

$\nabla^2 V = 4, \quad [1,2] \times [0,2]$

CC: consistentes con $(x-y)^2$

Analítica: $V = (x-y)^2$

E4 — Poisson fuente variable

$\nabla^2 V = \frac{x}{y} + \frac{y}{x}, \quad [1,2] \times [1,2]$

CC: $V(1,y) = y\ln y, \;\; V(2,y) = 2y\ln(2y),$
$V(x,1) = x\ln x, \;\; V(x,2) = 2x\ln(2x)$

Analítica: $V = xy\ln(xy)$

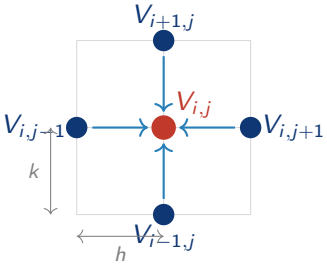
Discretización — Esquema de 5 Puntos

Malla uniforme: $h = \Delta x$, $k = \Delta y$

Diferencias centradas de orden 2:

$$\frac{\partial^2 V}{\partial x^2} \approx \frac{V_{i,j+1} - 2V_{i,j} + V_{i,j-1}}{h^2}$$

$$\frac{\partial^2 V}{\partial y^2} \approx \frac{V_{i+1,j} - 2V_{i,j} + V_{i-1,j}}{k^2}$$



Ecuación de actualización:

$$\boxed{V_{i,j} = \frac{(V_{i+1,j} + V_{i-1,j})k^2 + (V_{i,j+1} + V_{i,j-1})h^2 - f_{i,j}h^2k^2}{2(h^2 + k^2)}}$$

Error de discretización: $\mathcal{O}(h^2 + k^2)$ — **orden 2**

Iteración de Jacobi

Algoritmo iterativo

1. Inicializar: $V^{(0)} = 0$ en interior, CC en bordes
2. Calcular $V^{(n+1)}$ **completo** con valores de $V^{(n)}$
3. Medir convergencia:

$$\delta = \max_{i,j} \left| V_{i,j}^{(n+1)} - V_{i,j}^{(n)} \right|$$

4. Si $\delta < 10^{-6} \rightarrow$ **convergió**
5. Si $n = 400\,000 \rightarrow$ **detener**

Parámetros

Complejidad — $\mathcal{O}(N^4)$

Trabajo/iter:	$\mathcal{O}(N^2)$
Iters. totales:	$\mathcal{O}(N^2)$
$\rho \approx 1 - \frac{\pi^2}{2} h^2$	con $h = 1/N$
Total:	$\boldsymbol{\mathcal{O}(N^4)}$

Característica de Jacobi

Requiere **2 arreglos** en memoria simultáneos (V y V_{new}).

Fácilmente paralelizable.

Fortran — `gfortran -O2`

Características

- Arrays `allocatable real(8)`
- Storage: **column-major**
- Índice $i \to y$, $j \to x$
- Timer: `cpu_time()`

Por qué es el más rápido

Arrays **contiguos** + bucles DO → máxima localidad de caché.
-O2 vectoriza el inner loop.

```fortran
do it = 1, 400000
  delta = 0.0d0
  do i = 1, N-1
    do j = 1, M-1
      T_new(i,j) = (                          &
          (T(i+1,j)+T(i-1,j))*k2              &
        + (T(i,j+1)+T(i,j-1))*h2              &
        - source(i,j)*h2*k2                   &
        ) / denom
      delta = max(delta,                       &
                  abs(T_new(i,j)-T(i,j)))
    end do
  end do
  T = T_new
  if (delta < tol) exit
end do
```

C++ — g++ -O2

Características

- vector<vector<double>>
- Storage: **heap no contiguo**
- Timer: chrono
- Fuente via switch

Limitación

Acceso a V[i±1][j] sigue punteros no contiguos → **cache misses** en dirección y.

```
while (delta > TOLERANCE
       && iters < MAX_ITERATIONS) {
    delta = 0.0;
    V_old = V;
    for (int i = 1; i < M; ++i) {
      for (int j = 1; j < N; ++j) {
        double f = source_term(
                     x_ini+i*h,
                     y_ini+j*k);
        V[i][j] = (
          (V_old[i+1][j]+V_old[i-1][j])*(k*k)
         +(V_old[i][j+1]+V_old[i][j-1])*(h*h)
         - f*h2k2) / denom;
        delta = std::max(delta,
          std::abs(V[i][j]-V_old[i][j]));
      }
    }
    iters++;
}
```

Python/numpy

Características

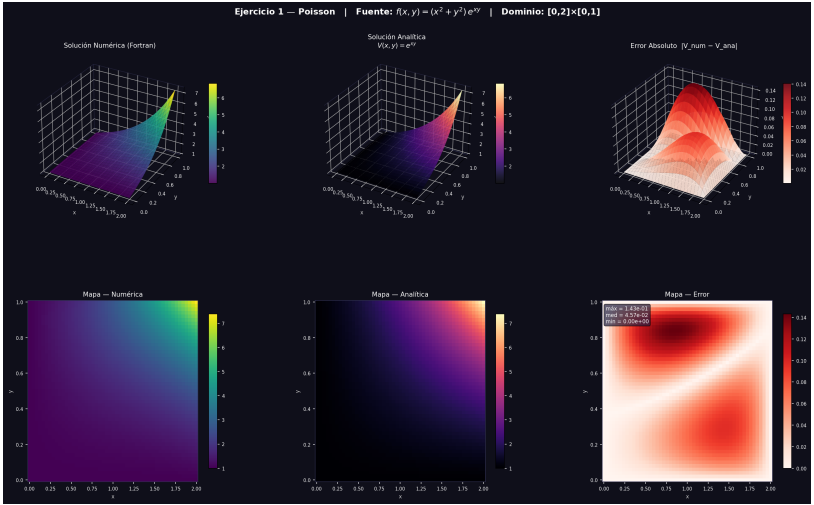
- Arrays numpy: **contiguos** (C row-major)
- **Sin bucles** Python en el inner loop
- Jacobi 100 % vectorizado
- Timer: `perf_counter()`

Por qué es competitivo

Slices de numpy ejecutan en **C nativo**.
Memoria contigua → sin cache misses.

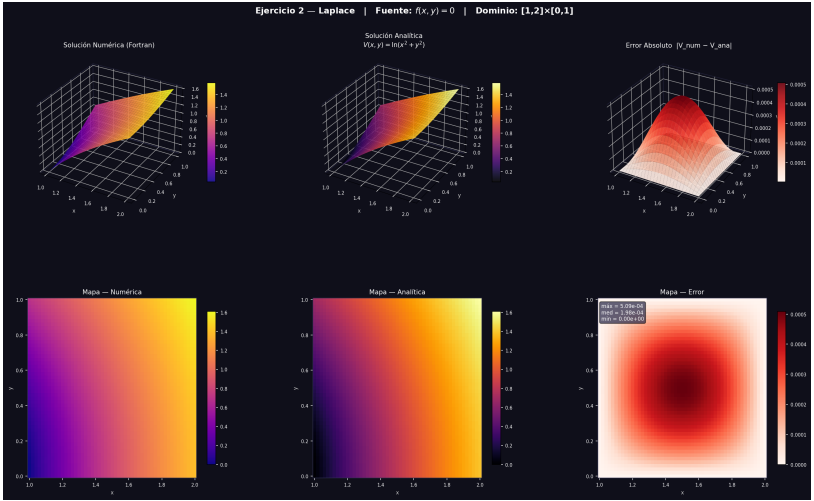
```
for it in range(1, MAX_ITER + 1):
    T_new = T.copy()

    T_new[1:-1, 1:-1] = (
        (T[2:,   1:-1] + T[:-2, 1:-1]) * k2
      + (T[1:-1, 2:]   + T[1:-1, :-2]) * h2
      - source[1:-1, 1:-1] * h2 * k2
    ) / denom

    delta = np.max(
        np.abs(T_new[1:-1,1:-1]
                - T[1:-1,1:-1]))
    T = T_new
    if delta < TOL:
        break
```

E1 — Poisson: $\nabla^2 V = (x^2 + y^2)e^{xy} \quad \Rightarrow \quad V = e^{xy}$



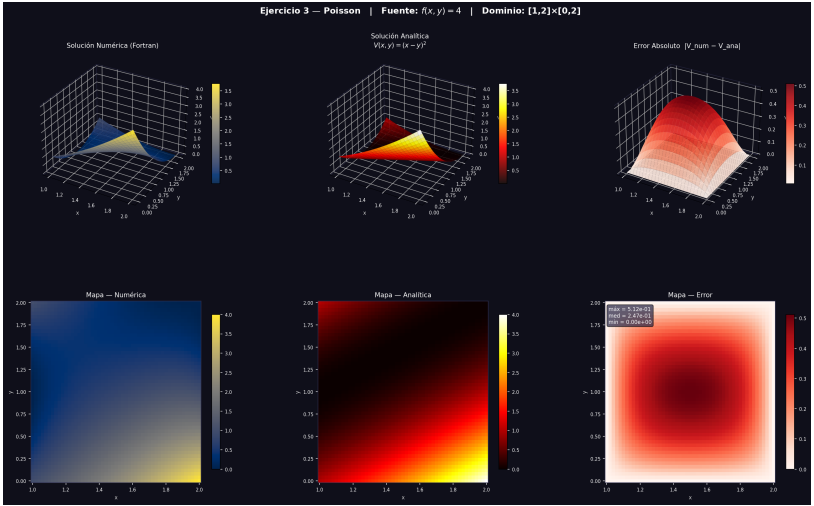
Malla 100×100 · **14 672 iteraciones** · Error máx. $1{,}4 \times 10^{-1}$ (gradiente alto en $x = 2, y = 1$)

E2 — Laplace: $\nabla^2 V = 0 \quad \Rightarrow \quad V = \ln(x^2 + y^2)$



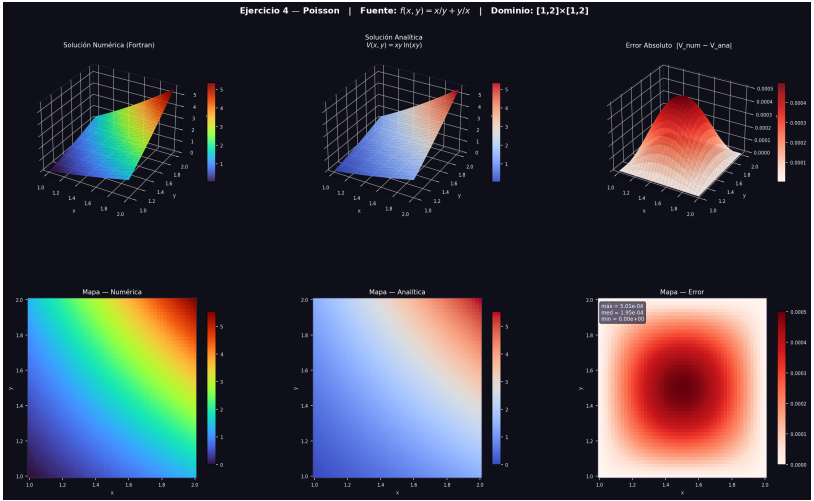
Malla 100×100 · **13 368 iteraciones** · Error máx. $2{,}0 \times 10^{-3}$ (solución suave, sin fuente interna)

E3 — Poisson: $\nabla^2 V = 4 \quad \Rightarrow \quad V = (x-y)^2$



Malla 100×100 · **13 220 iteraciones** · Error máx. $5{,}1 \times 10^{-1}$ (dominio no cuadrado, gradientes en

E4 — Poisson: $\nabla^2 V = x/y + y/x \quad \Rightarrow \quad V = xy\ln(xy)$



Malla 100×100 · **14 816 iteraciones** · Error máx. $2{,}0 \times 10^{-3}$ (solución suave en $[1,2]^2$)

Benchmark — Malla 100×100

Ejercicio	Fortran	C++	Python
E1 — Poisson e^{xy}	0.249 s	0.979 s	1.418 s
E2 — Laplace	0.185 s	0.361 s	1.292 s
E3 — Poisson $f = 4$	0.220 s	0.332 s	1.282 s
E4 — Poisson var.	0.245 s	0.410 s	1.431 s
Total	**0.90 s**	**2.08 s**	**5.42 s**

Speedup vs. Fortran

	Más lento
C++	2,3×
Python	6,0×

Convergencia

Todos convergieron.
Las ∼13 000–15 000 iter. necesarias están muy por debajo del límite de 400 000.

Benchmark — Malla 500 × 500

Ejerc.	Fortran	C++	Python	Iter.
E1	128.96 s	343.72 s	301.40 s	203 879
E2	67.70 s	122.00 s	299.44 s	171 155
E3	103.93 s	102.71 s	308.36 s	167 653
E4	130.79 s	146.96 s	401.51 s	207 350
Total	**431 s**	**715 s**	**1311 s**	—

Las celdas verdes señalan resultados sorpresivos.

¡Sorpresas!

E1: Python (301 s) supera a C++ (344 s)

E3: C++ (102 s) iguala a Fortran (104 s)

Speedup vs. Fortran

C++	1,66×
Python	3,04×

¿Por qué Fortran gana? — Caché y Memoria

Fortran

Array **contiguo** column-major

Bucles DO: acceso óptimo a caché L1/L2

-O2 vectoriza inner loop

Caché: óptimo

C++

`vector<vector>` heap fragmentado

Acceso a filas adyacentes: punteros dispersos

Cache misses en dir. y

Python/numpy

Array numpy C row-major

Slices ejecutan en C nativo SIMD en bloques

Caché: bueno

Fortran: 0.90 s $\xrightarrow{\times 2.3}$ **C++: 2.08 s** $\xrightarrow{\times 2.6}$ **Python: 5.42 s**

Python supera a C++ a 500×500 — ¿Por qué?

El cuello de botella de C++

`vector<vector<double>>` con $N = 500$:

- 500 filas = 500 punteros independientes
- Cada fila $\approx$ 4 KB en posición aleatoria de memoria
- Acceso a `V[i+1][j]`: probable **cache miss L2**
- Penalización **crece con** N

La ventaja de numpy

Array contiguo en memoria:

- `T[2:,1:-1]`: acceso secuencial por filas
- Operaciones **SIMD** sobre bloques contiguos
- A 500×500: cuello de botella = ancho de banda de memoria, no overhead de Python

E1 — 500×500: Fortran **128.96 s** C++ **343.72 s** Python **301.40 s**

Python es **12 % más rápido** que C++ gracias a la contigüidad de memoria

Escalado $\mathcal{O}(N^4)$ y Convergencia

Razón de tiempo: 100 → 500

	Obs.	Teórico
Fortran	481×	
C++	344×	347×
Python	242×	

Iter.: 203 879/14 672 = 13,9×

Trabajo/iter.: ×25 → 13,9 × 25 = 347×

Con 100 000 iter. → no convergía

Error residual ≈ 0,1–0,5 a 500 × 500.

Con 400 000 iter. → convergió

E1	203 879
E2	171 155
E3	167 653
E4	207 350

Margen disponible: ∼ 48 %

Para $N \geq 1000$: usar **SOR** o **Multigrid**.

Conclusiones

1. **Fortran es el más rápido** en casi todos los casos: 1,7–2,3× sobre C++ y 3–6× sobre Python. Debe su rendimiento al almacenamiento contiguo y a `gfortran -O2`.
2. **C++ no supera a Fortran** con `vector<vector<double>`. El layout fragmentado genera cache misses en dirección y. La solución: usar un arreglo plano 1D con indexación manual.
3. **Python/numpy supera a C++** en mallas grandes (E1, 500×500: 301 s vs 344 s) por acceso a memoria contiguo.
4. **Con 400 000 iteraciones Jacobi converge** a 500×500. Para mallas mayores se recomiendan SOR o multigrid.
5. **Los tres códigos son algorítmicamente equivalentes**: mismas iteraciones, mismos errores — las diferencias son solo de rendimiento.